

機器學習到大型語言模型

從最基礎的原理開始，一層一層往上疊，四個章節走完機器學習、深度學習、自然語言處理、大型語言模型。每個機制都先建立跟任何應用無關的地基，疊完才代入例子；圖表依內容性質挑選，不是每處都套同一種。

全書地圖	P.02
第一章 機器學習	P.03
第二章 深度學習	P.08
第三章 自然語言處理	P.12
第四章 大型語言模型	P.16

在開始之前

全書地圖：四個領域怎麼接在一起

這四個詞常常被混著用，但它們回答的是不同層次的問題。先把關係講清楚，後面四章才不會覺得是四個互不相干的主題。



四個箭頭代表四種不同的關係，不是單純的大圈圈包小圈圈：ML→DL 是「其中一種做法」；DL→NLP 是「把這種做法應用在某一種資料上」；NLP→LLM 是「這個應用領域裡目前最強的一種技術」。第一章到第四章，就是照這個順序把每一格拆開講。

為什麼照這個順序學

機器學習先建立最基本的詞彙——什麼叫「訓練」、什麼叫「模型學到東西」、怎麼知道學得好不好。這些詞彙在深度學習、NLP、LLM 裡會不斷重複出現，只是換了更複雜的外殼。深度學習講的是「怎麼讓機器自己找特徵，不用人工設計」。自然語言處理把這套技術落到「文字」這個特定的資料型態上，處理「電腦看不懂文字」這個根本問題。大型語言模型則是這條路走到現在最極端的結果：把 NLP 的技術跟深度學習的規模都推到最大。

每一章的內部順序也是同一個邏輯：先講一個跟任何案例都無關的最基礎原理，一步一步疊加元件，疊到完整機制後才代入例子，例子只是用來確認「這個機制套進真實情境長什麼樣子」，不是用來取代原理本身。

第一章 機器學習 Machine Learning

1.1 地基：機器「學習」到底在學什麼

傳統寫程式是人把規則寫死：如果 A 就做 B。機器學習反過來——不寫死規則，而是給機器大量「輸入、正確答案」的配對資料，讓它自己找出一個函數，之後看到新的輸入也能給出合理的答案。用最抽象的方式寫：給一堆 X (輸入) 跟 y (正確答案)，機器學習要找出一個函數 f ，使得 $f(X)$ 的結果盡量接近 y 。

這類「有正確答案可以對照」的學習方式叫**監督式學習 (Supervised Learning)**，是機器學習裡最基礎、應用最廣的一種。監督式學習底下再分兩種任務：

任務類型	要預測的東西	例子
迴歸 Regression	連續數值	房價、氣溫、股價
分類 Classification	離散類別	是不是垃圾郵件、圖片是貓還是狗

第一章大部分內容圍繞「分類」展開，因為分類問題最容易看懂機器學習的核心邏輯，而且後面深度學習、NLP、LLM 遇到的很多任務，本質上也是分類（例如「這個詞的詞性是什麼」「這段話是正面還是負面」）。

1.2 怎麼知道模型學得好不好：評估指標

底層概念。「模型的答案對不對」聽起來像是非黑即白的事，但拆開來看其實有四種結果，完全不需要牽扯任何真實案例，用最抽象的「篩檢」邏輯就能說清楚：

實際情況 \ 模型判斷	判斷為「是」	判斷為「不是」
實際上「是」	TP 真陽性（猜對）	FN 偽陰性（漏掉了）
實際上「不是」	FP 偽陽性（誤判）	TN 真陰性（猜對）

這張 2×2 表格叫**混淆矩陣 (Confusion Matrix)**，是幾乎所有分類問題評估的地基。從這四格可以疊出三個最常用的指標：

指標	公式	白話意義
準確度 Accuracy	$(TP+TN) / \text{全部}$	整體猜對的比例
精確率 Precision	$TP / (TP+FP)$	「判斷為是」的裡面，真的是的比例——判斷越挑剔，這個數字越高
召回率 Recall	$TP / (TP+FN)$	所有「真的是」的裡面，被抓到的比例——判斷越寬鬆，這個數字越高
F1-Score	Precision 與 Recall 的調和平均	兩者要兼顧時的綜合分數

Precision 跟 Recall 通常是拉鋸的：門檻設得越嚴格（越不輕易判斷為「是」），Precision 會上升但 Recall 會下降，反過來也一樣。這個拉鋸關係可以畫成一條曲線——**ROC 曲線**，橫軸是誤判率（FPR）、縱軸是抓到率（TPR），依不同門檻畫出整條曲線；曲線下的面積叫 **AUC**，0.5 代表模型等於亂猜，0.9~1.0 代表表現極佳。

資料要怎麼切

模型不能拿訓練時看過的資料來自誇「我猜對了」，這樣沒有意義，所以資料要切成「訓練用」跟「測試用」兩份，測試那份模型從沒看過。常見三種切法：

方法	做法	適合情境
訓練測試法	直接切一份（常見比例 75/25），訓練歸訓練、測試歸測試	資料量大
k 疊交互驗證 k-fold	資料切成 k 份，輪流用 k-1 份訓練、1 份測試，重複 k 次取平均	希望結果更穩定、不受單一切法運氣影響
自助法 Bootstrap	取後放回反覆抽樣，用抽到的資料訓練、沒抽到的資料測試	資料量非常小

1.3 決策樹：一步步問問題來分類

底層概念。先完全撇開決策樹，只問一個問題：一袋球，如果全部同一個顏色，你要摸出一顆猜顏色，會不會很有把握？如果一半一半兩種顏色呢？前者幾乎不用猜，後者猜錯機率最高。

這個「猜起來有多沒把握」的程度，叫**亂度 (Entropy)**——完全同一類 = 亂度最低；各類別比例越平均 = 亂度越高。這個地基跟決策樹、跟任何資料集都還沒有關係，純粹是「一堆東西的混亂程度」這件事本身。

疊加：用亂度來決定怎麼分裂

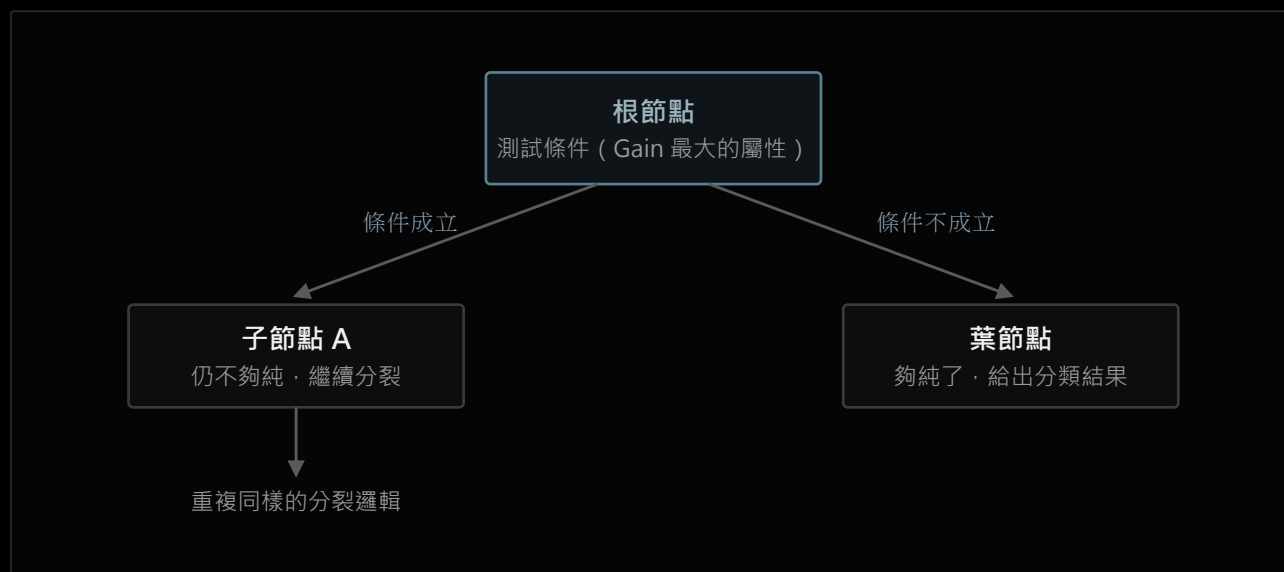
決策樹由節點（測試條件）、分支、葉節點（最終分類結果）組成，用「貪婪演算法」由上而下遞迴分割：每一步都選一個屬性，把資料切成兩邊，目標是切完之後兩邊都盡量「純」（亂度低）。怎麼選那個屬性？計算**資訊獲利 (Information Gain)**：

亂度公式： $I(p,n) = -\sum (p_i/(p+n)) \cdot \log_2(p_i/(p+n))$

資訊獲利： $Gain(A) = \text{分裂前的亂度} - \text{分裂後的加權亂度}$

規則：選 $Gain$ 最大的屬性優先分裂（代表分裂後最能把資料變「純」）

另一種同樣衡量「不純度」的算法是 **Gini 索引**： $Gini(D) = 1 - \sum p_j^2$ ，常搭配連續數值屬性使用（選一個切點，讓切完後加權 Gini 值最小），概念跟亂度一樣，只是公式不同。



決策樹本身就是一棵樹：每個節點問一個問題，資料依答案分成兩邊，一路分裂到某個節點「夠純」（亂度夠低）為止，變成葉節點給出結果。

如果一直分裂到每個葉節點都完美純淨，樹會長得又深又細，把訓練資料的雜訊也背下來，遇到新資料反而表現差——這叫**過擬合 (Overfitting)**。解法是修剪：**預先修剪**（分裂過程中提前喊停，例如限制最大深度）或**事後修剪**（先長成完整的樹，再剪掉沒有幫助的分支）。

1.4 集成學習：讓很多個「還可以」的模型變成一個「很準」的模型

單一棵決策樹容易不穩定（資料稍微變一點，樹可能長得完全不一樣）。集成學習的想法是同時訓練很多個「準確率只要贏過亂猜」的弱模型，再把它們的意見合起來，通常會比任何一個單獨的模型更準更穩。兩條主要路線：

	BAGGING	BOOSTING
怎麼產生多個模型	對資料取後放回反覆抽樣，各自獨立訓練	每一輪都針對前一輪答錯的樣本加強學習
模型之間的關係	互相獨立，平行訓練	一個接一個，後面的模型修正前面的錯
合併方式	投票（分類） / 平均（迴歸）	加權投票，準確率越高的模型權重越大
代表演算法	Random Forest （= Bagging + 決策樹，連特徵欄位也一起抽樣）	AdaBoost （提高被誤判樣本的權重，讓下一個模型專注在難分的樣本上）
怕什麼	不太怕雜訊，多顆樹平均掉個別誤差	雜訊資料會讓後面的模型過度關注在錯誤樣本上

1.5 距離與邊界：KNN、SVM、貝氏分類器（概念性理解即可）

這三個方法運作邏輯跟決策樹、集成學習完全不同，各自抓住一個直覺就夠用：

方法	核心直覺	優點	缺點
KNN K-最近 鄰居	看離新樣本最近的 K 個鄰居，多數決決定類別。K 越小邊界越鋸齒（容易過擬合），K 越大邊界越平滑	邏輯簡單直覺，不用真的「訓練」模型	資料量大時，每次預測都要跟全部資料算距離，速度慢；使用前須做特徵標準化
SVM 支持向 量機	在兩類別中間找一條間隔最大的分界線（超平面），最靠邊界的樣本點叫「支持向量」	邊界清楚、理論扎實；搭配 Kernel trick 可以處理非線性可分的資料（映射到更高維度）	資料量大時訓練變慢，Kernel 選擇需要實驗
貝氏分類器 Naive Bayes	用貝氏定理 $P(C X)=P(X C)P(C)/P(X)$ ，假設各特徵互相獨立，算出屬於每個類別的機率，選機率最高的	訓練速度快，資料量小也能用；文字分類效果好（搭配 TF-IDF）	「特徵互相獨立」的假設在真實資料裡常常不成立，但實務上效果仍常常不錯

第二章 深度學習 Deep Learning

2.1 為什麼需要深度學習

第一章的方法（決策樹、SVM...）有一個共同前提：要先由人決定「用哪些特徵」。要分辨貓狗，得先想好「耳朵形狀」「毛色」這些特徵欄位，這個過程叫**特徵工程**，非常仰賴人的經驗，而且很多資料（例如一張圖片的原始像素、一段語音的原始波形）根本不知道該怎麼手動設計特徵。

深度學習用「多層神經網路」取代人工特徵工程：資料直接餵進去，網路自己在訓練過程中學出該注意哪些特徵、怎麼組合這些特徵。一個神經網路由輸入層、隱藏層（超過 2 層才稱得上「深度」）、輸出層組成，每個神經元把輸入加權加總，再通過一個**活化函數**（如 ReLU、sigmoid）做非線性轉換——如果沒有這個非線性轉換，疊再多層本質上還是等於一層線性函數，疊不出複雜的判斷能力。

2.2 地基：梯度下降怎麼調整參數

底層概念，先完全不提神經網路。國中數學教過斜率：函數 $y = x^2$ 在某一點的切線斜率，代表「這個點如果 x 稍微增加一點， y 大概會怎麼變」。用具體數字看：

x	$y = x^2$	這一點附近的斜率	意義
1	1	約 2	x 增加， y 會跟著明顯增加
0.1	0.01	約 0.2	x 增加， y 幾乎不太變
3	9	約 6	斜率本身也會隨位置改變

這就是梯度的地基：斜率告訴你「往哪個方向調整，函數值會變小（或變大）」。深度學習裡「函數值」換成了 **Loss**（誤差，模型猜的跟正確答案差多少），「 x 」換成了模型的參數（權重），於是有了**梯度下降（Gradient Descent）**：不是解方程式直接算出最佳參數，而是像逼近一樣，每一步都朝著讓 Loss 變小的方向，把參數挪一小步，重複很多輪。用具體數字示範一次（單一參數 w ，簡化過的例子）：


```
第 1 輪：w = 0.50  loss = 8.20
第 2 輪：w = 0.31  loss = 5.10    (沿梯度方向移動一步，loss 下降)
第 3 輪：w = 0.19  loss = 2.94
第 4 輪：w = 0.14  loss = 1.68
第 5 輪：w = 0.12  loss = 1.35    (loss 下降幅度變小，接近收斂)
```

每一步移動的幅度由**學習率 (learning rate)** 決定：太大會在最佳點附近來回震盪、甚至越調越糟；太小則要跑非常多輪才會收斂。當網路疊得很深、又用 sigmoid 這類活化函數時，梯度在一層層反向傳遞的過程中會越乘越小，前面幾層幾乎學不到東西，這叫**梯度消失 (Vanishing Gradient)**，這也是現代深度網路多半改用 ReLU 的原因之一。

訓練的兩個階段

神經網路的訓練分兩個方向：**正向傳播** (資料一路往前算出預測值跟 Loss)、**反向傳播** (依微積分的鏈式法則，從 Loss 往回對每一層的參數算偏微分，也就是梯度)，公式是 $w^+ = w - \eta \cdot \partial E / \partial w$ (η 是學習率)。

2.3 PyTorch 的基本語言：Tensor 與 Autograd

Tensor 是 PyTorch 的基本運算單元，概念上就是 NumPy 的多維陣列，差別是 Tensor 可以放到 GPU 上加速運算。**Autograd (自動微分)** 是 PyTorch 幫你自動算梯度的機制：把一個 Tensor 的 `requires_grad` 設成 True，之後這個 Tensor 參與的每一步運算都會被記錄成一張「計算圖」；呼叫 `.backward()` 後，PyTorch 就沿著這張圖反向傳播，自動把每個參數的梯度算好、存進 `.grad`。不需要梯度的地方 (例如推論階段) 可以用 `.detach()` 或包在 `with torch.no_grad()` 裡跳過記錄，省下計算跟記憶體開銷。



PyTorch 屬於「動態計算圖」：圖是程式實際執行時才建立的，每次迭代都重新建一次，換來的是可以用一般 Python 的 if/for 控制流程，彈性很高。

2.4 組出一個神經網路：nn.Module 與訓練迴圈

PyTorch 用 `nn.Module` 定義網路：繼承這個類別，在 `__init__` 裡宣告要用到的層，在 `forward()` 裡定義資料怎麼流過這些層。常見元件：`nn.Linear`（線性變換 $wx+b$ ）、`nn.ReLU`（非線性活化）、`nn.Flatten`（把多維資料攤平）、`nn.Sequential`（把多層包成一個容器依序執行）、`nn.Softmax`（把輸出轉成加總為 1 的機率分布，常用在多分類的最後一層）。

損失函數：怎麼量化「猜得有多差」

損失函數	常用場景
MSE (均方誤差)	迴歸任務
Cross Entropy (交叉熵)	分類任務，計算與梯度表現比 MSE 更好，收斂較快
Hinge Loss	SVM 系列方法在用

優化器：知道梯度之後，具體怎麼調參數

優化器	邏輯
SGD	$\text{weight} = \text{weight} - \text{lr} \times \text{gradient}$ ，最基本版本
Momentum	模擬物理慣性：同方向持續調整就加速，方向反覆就減速，避免震盪
AdaGrad	依每個參數的歷史梯度大小自動調整各自的學習率，缺點是訓練後期容易停滯
Adam	結合 Momentum 與 AdaGrad 的優點，是目前最常用的預設選擇（常見預設 $\text{lr}=0.001$ ）

標準訓練迴圈



五個步驟每個 batch 都要重複一次；PyTorch 預設梯度會累加，所以每一輪開頭一定要先 `zero_grad()` 清空，否則會跟上一輪的梯度疊在一起。

超參數（人工設定、不是模型自己學出來的）包括：**Batch Size**（一次丟多少筆資料）、**Epoch**（整個訓練集完整跑過一輪的次數）、網路的層數與神經元數量。

Dropout：一種簡單有效的正則化

訓練時以某個機率 p ，隨機把部分神經元的輸出關閉（暫時設為 0），避免模型過度依賴少數神經元、降低過擬合，效果上相當於同時訓練了很多個共享權重的子網路。推論階段要打開全部神經元，並把權重乘上保留機率做等比例縮放——對應到 PyTorch 就是呼叫 `model.eval()` 切換到評估模式。

第三章 自然語言處理 NLP

3.1 電腦要理解語言，得先解決哪些問題

人類語言對電腦來說是一長串沒有明確結構的符號。NLP 要把這串符號拆解成電腦能處理的元件，常見的幾個子任務：

子任務	在做什麼
分詞化 Tokenization	把句子切成更小的單位（詞、子詞、標點）
詞幹提取 / 詞形還原	把詞還原成基礎形式，例如 scheduling → schedule
詞性標記 POS Tagging	標註每個詞是名詞、動詞...
命名實體識別 NER	找出文本裡的人名、地名、組織名
情感分析	判斷語氣是正面、負面或中性
情境理解	依上下文解讀單詞或短語的實際含義

這些子任務過去各自需要獨立的模型或規則來處理，而現代的語言模型（尤其是第四章要講的 LLM）某種程度上是用同一個模型、同一套機制，把這些子任務都吃下去了。

3.2 語言模型的地基：預測下一個字的機率

底層概念，先不管模型多大多小。把「今天真開心，外面的天氣很 _ _」丟給任何一個人，大多數人會填「好」「棒」，幾乎不會填「爛」「濫」。你的大腦其實在做一件事：對所有候選字，各自估計一個「這裡填這個字合理嗎」的機率。

候選字	直覺機率
好	0.40
棒	0.30
差	0.05
爛	0.02

語言模型做的事，本質上就是不斷重複這個機率預測的動作——只是它是用第二章學的神經網路，從大量文字裡訓練出來估計這個機率分布，而不是靠人的直覺。訓練資料的品質會直接影響預測品質：訓練資料裡英文的比例高，模型的英文回覆通常也會比中文更流暢，就是這個道理。

依照「怎麼利用上下文」，語言模型大致分兩種取向（第四章的 GPT、BERT 會分別對應到這兩種）：

取向	邏輯
自迴歸 Autoregressive	只看前面已經出現的字，逐字預測下一個字，像文字接龍
自編碼 Autoencoding	可以同時看前後文，去猜句子中被遮住的字

3.3 分詞：Tokenization

模型看到的最小單位叫 **Token**，一個 Token 不一定等於一個字，可能是一個單字、一個子詞、甚至一個標點符號，實際怎麼切取決於訓練模型時用的分詞策略。中文尤其麻煩：「台灣科技大學」可能被切成「台灣 / 科技 / 大學」，也可能是「台灣科技 / 大學」，沒有唯一正確答案。

Token 化的粒度甚至會影響模型的能力：把 lollipop 反過來拼，如果整個字是一個 token，模型很難做對；但如果每個字母之間都加上連字號（l-o-l-l-i-p-o-p），每個字母變成獨立 token，模型反而能穩定做對——這說明 token 的切法會直接決定模型「看得到」什麼細節。

常見的分詞演算法：

方法	核心原理	用在哪
BPE Byte Pair Encoding	反覆合併最常出現的相鄰字元對，直到達到預設的詞彙表大小； 能把罕見詞拆成常見子詞，解決「詞典裡沒這個詞」的問題	GPT 系列、 BERT
SentencePiece	把空格也當成一個普通字符，直接在字元層級分詞，不依賴人工 先斷詞規則，適合處理多國語言	T5 等 Google 的模型

Token 除了是模型計算的基本單位，也是 API 計費的單位——輸入加輸出的 token 數量，直接決定一次呼叫的成本跟長度上限。

3.4 詞嵌入：Word Embedding

底層概念：文字要先變成數字，電腦才能算。最直覺的做法是 **one-hot encoding**：每個字對應一個很長的向量，只有自己的位置是 1，其他全部是 0。這個做法有兩個明顯的問題：

1. **無法辨別一詞多義。**「cold」在「寒冷」跟「感冒」兩種意思裡，one-hot 給出的向量完全一樣，等於告訴模型這兩個意思是同一回事。
2. **維度爆炸。**詞彙表有 16000 個詞，一個字就要用長度 16000 的向量表示，一篇 100 字的文件就是 (100, 16000) 的矩陣，多數位置都是浪費的 0，運算跟儲存都吃不消。

疊加：用 Embedding 取代 one-hot。Embedding 本質上是一個「降維」技術：把每個 token 對應到整數 ID，再透過一個 Embedding Layer，映射成一個較短的高維度向量（例如 768 或 1024 維，但遠比 16000 短）。這個向量不是隨便給的，而是透過神經網路訓練出來的：語義相近的詞（例如「狗」和「貓」），在這個向量空間裡的位置會彼此接近；向量甚至可以做語義運算，經典例子是「國王 - 男人 + 女人 $\approx$ 女王」。

怎麼衡量兩個向量「像不像」：餘弦相似度

向量之間最基本的相似度算法是**內積 (dot product)**：對應維度相乘再全部加總，數字越大代表方向越接近。但內積會被向量本身的長度影響（長向量內積容易偏大），所以語義相似度更常用**餘弦相似度 (Cosine Similarity)**——把內積除以兩個向量長度的乘積，等於把內積「正規化」成只看方向、不看長度：

情況	餘弦相似度	意義
方向完全一致	1	語義最相似
方向互相垂直	0	沒有關聯
方向完全相反	-1	語義最不相似

這一節建立的「向量、內積、相似度」這三個概念，是第四章 Attention 機制的地基——Attention 要做的核心計算，正是這裡的內積再進一步正規化。

第四章 大型語言模型 LLM

4.1 Transformer 架構與 Self-Attention

底層概念（延續第三章已經建立的向量與內積）。把文字轉成向量之後，「這兩個詞有多相關」這件事，可以直接用向量內積來算——這正是上一章 3.4 節的內容，只是現在要把它組裝成一整套機制。

向量 $a=(1,2)$ 、向量 $b=(2,1) \rightarrow a \cdot b = 1 \times 2 + 2 \times 1 = 4$ 分數高，方向接近，兩者「有關」
向量 $a=(1,2)$ 、向量 $c=(-2,1) \rightarrow a \cdot c = 1 \times (-2) + 2 \times 1 = 0$ 分數為 0，兩者「無關」

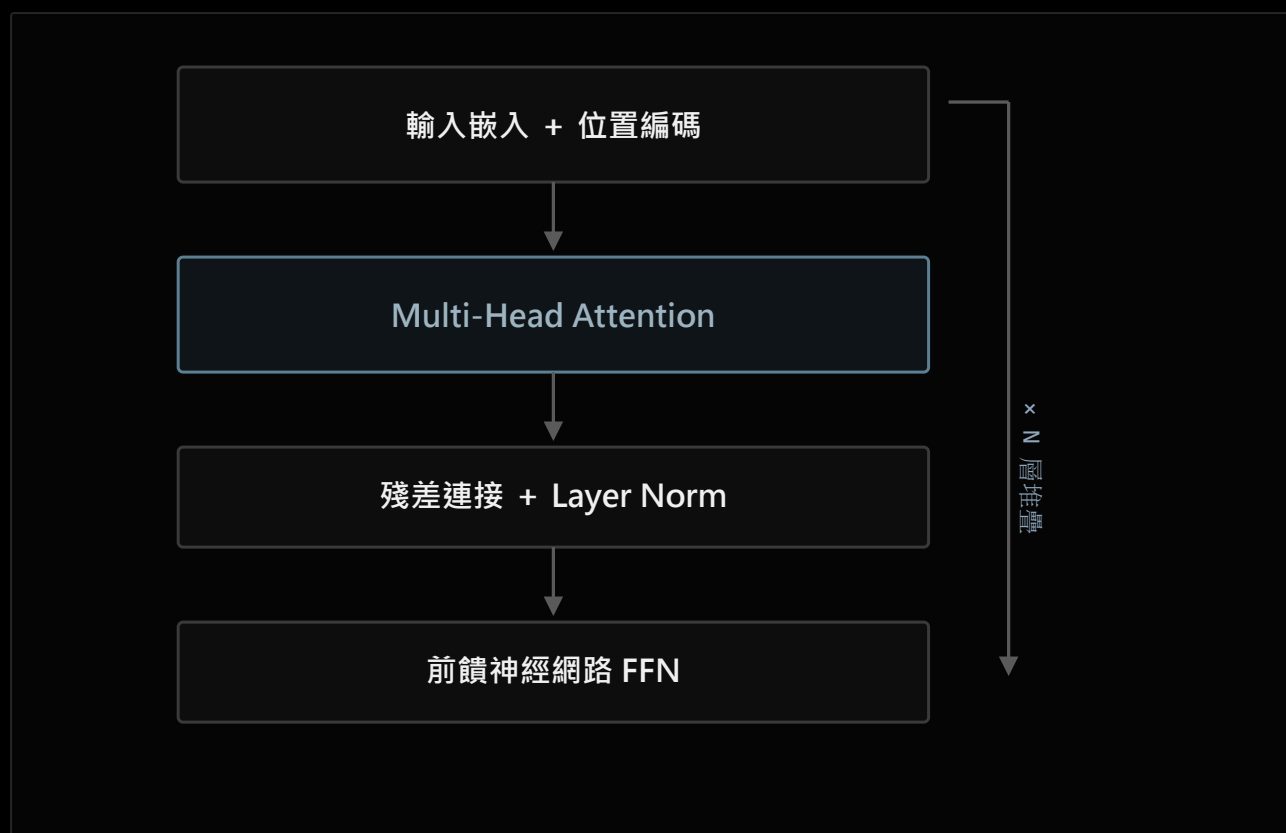
疊加：五個元件組成 Self-Attention



Query = 要去比對的向量；Key = 被比對的向量；Value = 比對完後真正要拿走的内容。跟單純的關鍵字比對不同：這裡每個位置都會被納入計算，只是依相似度分數拿到不同的權重——這種「連續的加權平均」而不是「非黑即白的篩選」，正是 Transformer 能處理模糊語意關係的原因。

一個詞往往同時跟好幾個不同面向的詞有關（例如指代關係、語氣、主題），只算一組 Q/K/V 只能抓到一種相關性，所以實務上會把 Q/K/V 拆成多組，各自獨立做一次上面的計算，最後把結果併起來，這叫**多頭注意力（Multi-Head Attention）**——概念上是同時從好幾個不同角度去看同一句話。另外，Attention 本身不知道詞的先後順序（所有位置都是平行計算的），所以還需要額外加上**位置編碼（Positional Encoding）**，把「這是第幾個字」的資訊也塞進向量裡。

整體架構：Attention 只是其中一塊



上一張圖的五格計算，其實只佔一層裡的一小塊；一層還包含殘差連接、Layer Norm、前饋神經網路，整個區塊再往上疊 N 次（例如 GPT-3 疊了 96 層），才是完整的 Transformer。

4.2 三大架構：GPT、BERT、T5

Transformer 原始架構包含 Encoder（編碼器）跟 Decoder（解碼器）兩部分。後續的模型依「用了哪部分」分成三條路線：

	GPT	BERT	T5
架構	Decoder-only	Encoder-only	Encoder-Decoder (完整版)
取向	自迴歸 (只看前文)	自編碼 (雙向 , 能看前後文)	兩者皆有
預訓練任務	逐字接龍 , 預測下一個字	MLM (隨機遮住 15% 的詞 , 猜被遮住的詞) + NSP (判斷兩句話是否連貫)	把所有任務統一成「文字轉文字」格式 , 前面加任務前綴 (如 "summarize: ")
擅長	文字生成、對話	語意理解、分類	翻譯、摘要等序列轉換任務

GPT 生成文字時，怎麼從機率分布裡選出下一個字，也有幾種常見策略：**Greedy Search**（每步都選機率最高的字，容易單調重複）、**Beam Search**（同時保留多條候選序列，最後選整體機率最高的）、**Top-K / Top-P 採樣**（只在機率較高的候選集合裡隨機選，比貪婪法更有變化）、**Temperature**（調整機率分布的平滑程度，越高越有創意，越低越保守）。

4.3 怎麼把模型訓練出來：預訓練、微調、對齊

把 3.2 節建立的「機率預測」地基往上疊三層，每一層都是在調整同一個機率分布，只是目的不同：



三個階段都是在調整「預測下一個字」的機率分布：預訓練學語言本身，SFT 學回答問題的格式，對齊學人類偏好的回答方式。

預訓練階段，不同架構在「猜什麼」上也不一樣：GPT 用 **CLM**（只看左邊，猜下一個字）、BERT 用 **MLM**（猜被遮住的字）、T5 用 **Span-corruption**（挖空一整段文字，猜整段）。

RLHF：三步驟把模型調整成人類偏好的樣子



對齊追求三個標準：有用性（Helpfulness）、誠實性（Honesty）、無害性（Harmlessness）。挑戰在於人工標註成本高、標註者本身可能有偏見，而且對齊後模型在某些原本擅長的任務（如翻譯）上，表現有時反而會下降，這個現象稱為 Alignment Tax。

第③步用的 **PPO（Proximal Policy Optimization）** 來自強化學習：把語言模型本身當作要學習的「策略」，獎勵模型打的分數當作「獎勵」，每次更新時額外加一個跟原始 SFT 模型的差異懲罰（KL 散度），限制模型不要為了拿高分而偏離原本的語言能力太遠。這一整套「用獎勵訊號調整策略」的邏輯，來自強化學習裡「Agent 依據 Reward 調整行為」的基本框架，PPO 只是這個框架裡效率較好、較穩定的一種優化演算法。

DPO：一個更簡單的替代方案

RLHF 需要同時維護、訓練四個模型（策略模型、價值模型、獎勵模型、參考模型），流程複雜、訓練也容易不穩定。**DPO（Direct Preference Optimization）** 換一個做法：直接拿「哪個答案比較好、哪個比較差」的成對資料，用一個損失函數同時拉高好答案的機率、壓低壞答案的機率，只需要 2 個模型（訓練中的模型 + 一個不變動的參考模型防止遺忘），把原本 2~3 個訓練階段簡化成 1 個，訓練也更穩定。

4.4 怎麼實際使用一個 LLM

有了一個訓練好、對齊好的 LLM 之後，實務上有幾種讓它「為你所用」的方式，各自的成本跟適合情境不同：

方式	做法	成本	適合情境
Prompt Engineering	純粹透過下指令、給範例引導模型輸出	最低	任務簡單、需要快速迭代
RAG 檢索增強生成	先用檢索技術找到正確資料，再交給模型生成回答	中等	需要專業知識、避免模型憑空亂答
Fine-Tuning	用特定任務的資料集微調模型參數	較高（需要標註資料）	希望模型的語氣、專業度整個固定下來
自己訓練模型	從零開始訓練一個全新模型	極高	只有極少數大型企業能負擔

四種方式沒有絕對的優劣，取捨看的是資料量、預算、還有需求會不會經常變動——這也是全書從第一章一路走到這裡想建立的能力：不是背下每個技術的名字，而是看到一個新問題時，知道該往哪個方向去想。